

# Master's Seminar: Artificial Intelligence - Visual Computing

## 3D Gaussian Splatting

Sebastian Frehmel

Artificial Intelligence Group,  
Fernuniversität Hagen, Germany

September 2nd, 2026



- ▶ **The NeRF Dilemma**

Neural Radiance Fields (NeRFs) are either slow and provide photorealistic scenes or they are reasonably fast but have to sacrifice image quality. Real-time applications are not possible.

- ▶ **3D Gaussians as a Novel Approach**

Using 3D Gaussians as the differentiable, explicit base primitive, and abandoning neural networks was the key to unlocking previously unseen rendering speeds.

- ▶ **Impact on the field of 3D Reconstruction**

Real-time rendering of reconstructed scenes represents a disruption in 3D Reconstruction enabling many new downstream applications.

- 1 Background
- 2 3D Gaussian Data Structure
- 3 Training Procedure
  - Point Cloud Initialization
  - Forward Pass
  - Backward Pass
- 4 Novel-View Synthesis
- 5 Summary and Discussion

## Core Concepts

### 3D Scene Creation

Novel-View Synthesis

Splatting

## Definitions

Creates 3D environments such as individual objects or outdoor spaces from an overlapping set of 2D input images.

## Core Concepts

3D Scene Creation

**Novel-View Synthesis**

Splatting

## Definitions

Synthesizes new 2D imagery from the 3D scene using new camera positions.

Real-time capability requires at least 24–30 frames per second (FPS) to appear smooth enough for the human eye.

## Core Concepts

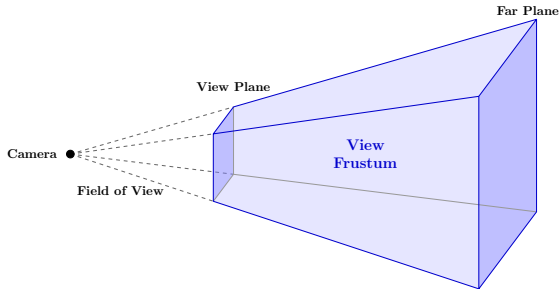
3D Scene Creation

Novel-View Synthesis

## Splatting

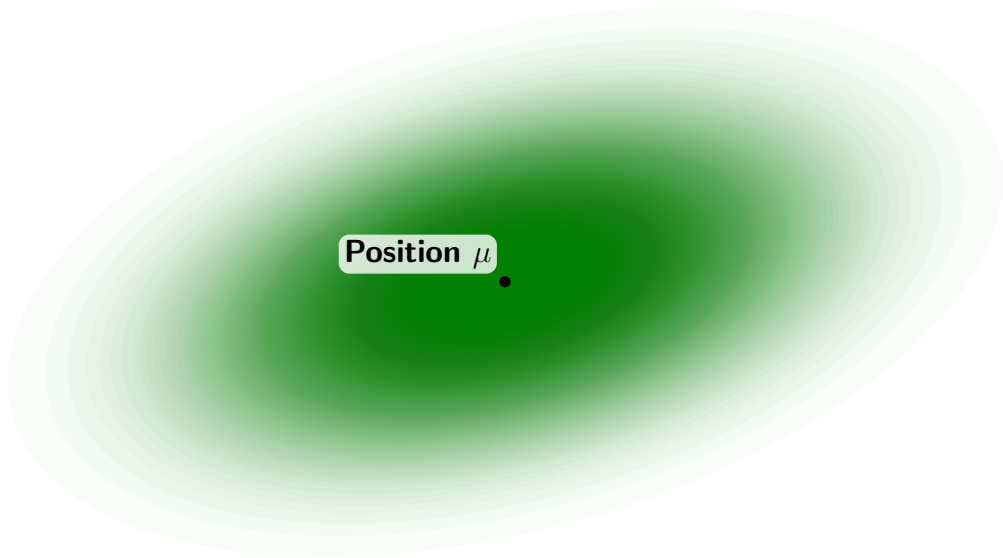
## Definitions

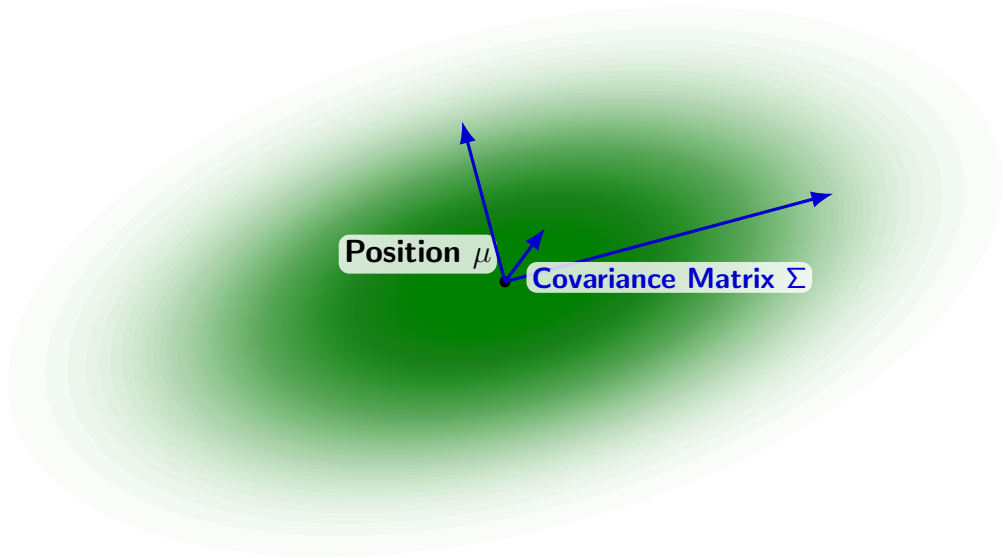
The process of moving all elements inside the view frustum onto the view plane preserving order, adding to aggregated color and opacity of each pixel.

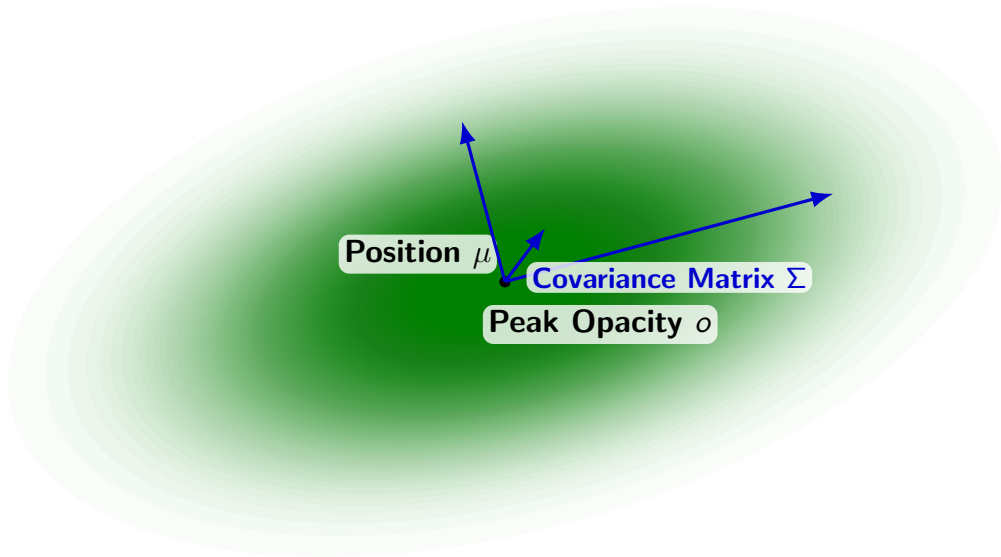


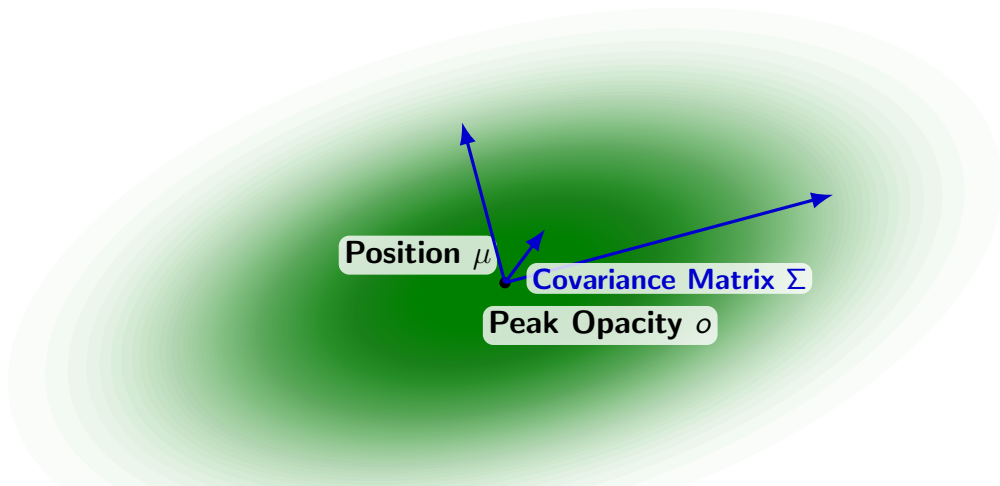
- 1 Background
- 2 3D Gaussian Data Structure
- 3 Training Procedure
  - Point Cloud Initialization
  - Forward Pass
  - Backward Pass
- 4 Novel-View Synthesis
- 5 Summary and Discussion



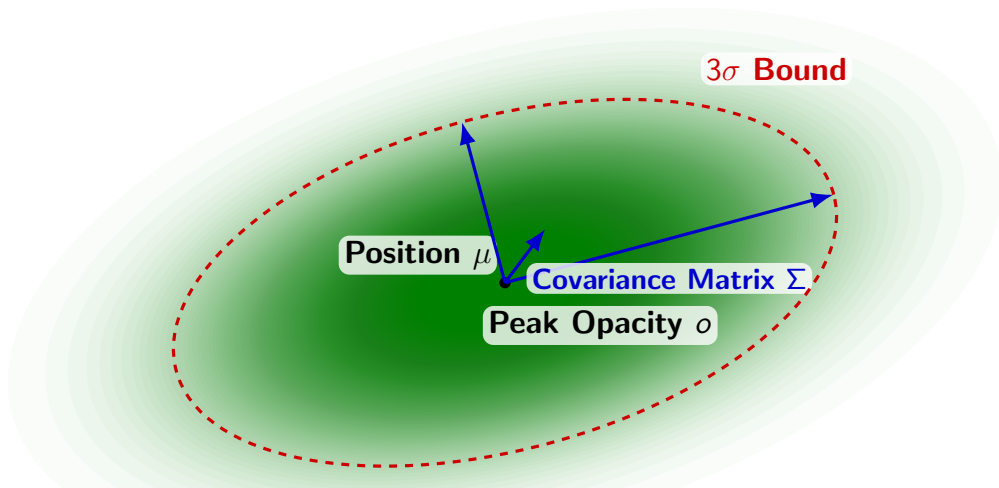




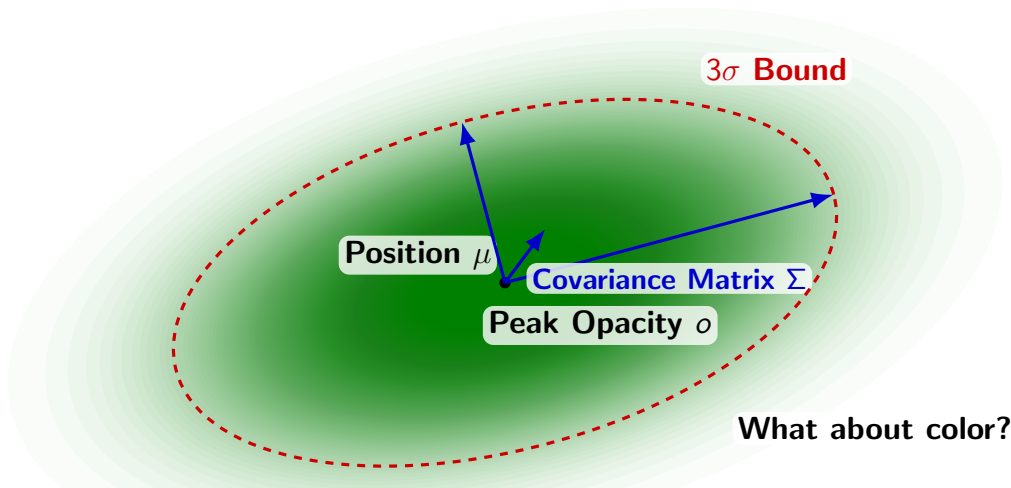




Multivariate Normal Distribution:  $\alpha(p) = o \cdot \exp\left(-\frac{1}{2}(p - \mu)^T \Sigma^{-1}(p - \mu)\right)$



$$\text{Multivariate Normal Distribution: } \alpha(p) = o \cdot \exp\left(-\frac{1}{2}(p - \mu)^T \Sigma^{-1}(p - \mu)\right)$$



Multivariate Normal Distribution:  $\alpha(p) = o \cdot \exp\left(-\frac{1}{2}(p - \mu)^T \Sigma^{-1}(p - \mu)\right)$

## Mathematical Setup

### The Validity Problem

Core Decomposition

Decomposed Components

Guaranteed Validity

## Details

To work with gradient descent, the covariance matrix  $\Sigma$  must be differentiable and remain positive semi-definite (eigenvalues  $\geq 0$ ) to be geometrically valid.

Running a loop during backpropagation to find valid values is computationally too costly.

## Mathematical Setup

The Validity Problem

## Core Decomposition

Decomposed Components

Guaranteed Validity

## Details

Kerbl et al. bypassed this by decomposing  $\Sigma$  into a scaling matrix  $S$  and rotational matrix  $R$  mirroring standard eigendecomposition ( $\Sigma = Q\Delta Q^T$ ):

$$\Sigma = RSS^T R^T \text{ with } \Delta = S^2, Q = R$$

## Mathematical Setup

The Validity Problem

Core Decomposition

## Decomposed Components

Guaranteed Validity

## Details

The diagonal scaling matrix  $S$  is further decomposed into three distinct scaling values. These represent the roots of the eigenvalues of  $\Sigma$ .

The rotational matrix  $R$  is decomposed into a four-value *quaternion*.

## Mathematical Setup

The Validity Problem

Core Decomposition

Decomposed Components

**Guaranteed Validity**

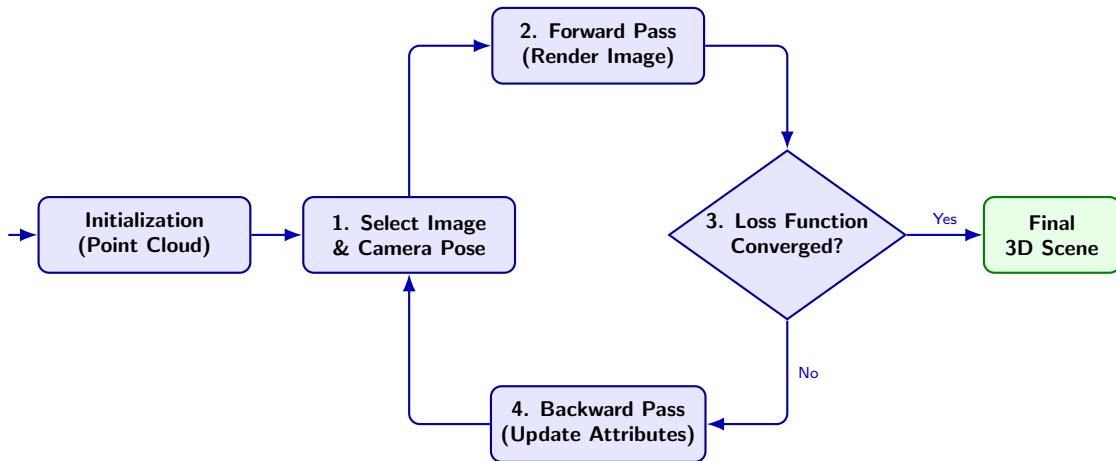
## Details

3DGS only stores and optimizes these 7 specific values for each 3D Gaussian.

Because an eigendecomposition is always positive semi-definite, constructing a new covariance matrix from these components is guaranteed to be geometrically valid.

- 1 Background
- 2 3D Gaussian Data Structure
- 3 Training Procedure**
  - Point Cloud Initialization
  - Forward Pass
  - Backward Pass
- 4 Novel-View Synthesis
- 5 Summary and Discussion

# The Training Cycle



- 1 Background
- 2 3D Gaussian Data Structure
- 3 Training Procedure**
  - Point Cloud Initialization
    - Forward Pass
    - Backward Pass
- 4 Novel-View Synthesis
- 5 Summary and Discussion

## Initialization Steps

### Structure from Motion

Camera Matrix

Primitive Initialization

## Details

The 3DGS algorithm requires initial geometry, which is commonly derived from a set of input images.

Because camera poses are usually unknown, Structure from Motion (SfM) software like COLMAP is used to solve a geometric optimization problem, providing an accurate 3D point cloud.

## Initialization Steps

Structure from Motion

**Camera Matrix**

Primitive Initialization

## Details

As another result from SfM, each input image is assigned a  $3 \times 3$  camera matrix  $W_{rot}$  which contains the camera's orientation in world space.

## Initialization Steps

Structure from Motion

Camera Matrix

## Primitive Initialization

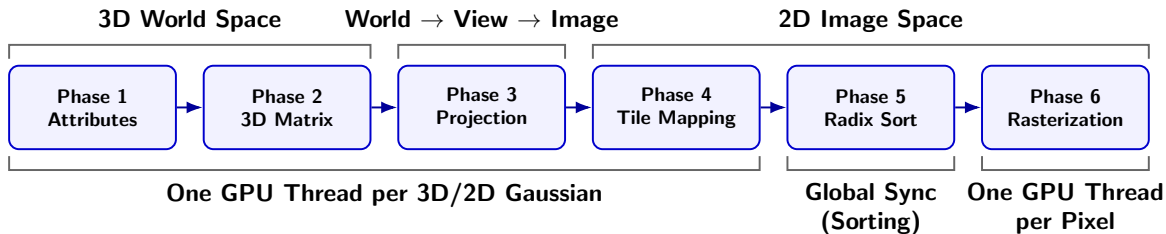
## Details

The SfM points act as the origins for the initial 3D Gaussians.

- ▶ They are initialized as perfect spheres.
- ▶ Radii are deduced from their three nearest neighbors.
- ▶ Opacity is set to 0.1 and color to any RGB value.

- 1 Background
- 2 3D Gaussian Data Structure
- 3 Training Procedure**
  - Point Cloud Initialization
  - **Forward Pass**
  - Backward Pass
- 4 Novel-View Synthesis
- 5 Summary and Discussion

# Forward Pass: Pipeline Architecture



### Phase 1: Preparing Constituent Attributes

- ▶ The GPU is running one GPU thread per 3D Gaussian.
- ▶ Raw values optimized during backpropagation are converted.
- ▶ Opacity is bounded to  $(0, 1)$  using a sigmoid function, scale uses the exponential function, and the rotational quaternion is normalized.

### Phase 1: Preparing Constituent Attributes

- ▶ The GPU is running one GPU thread per 3D Gaussian.
- ▶ Raw values optimized during backpropagation are converted.
- ▶ Opacity is bounded to  $(0, 1)$  using a sigmoid function, scale uses the exponential function, and the rotational quaternion is normalized.

### Phase 2: Reconstructing the 3D Covariance Matrix

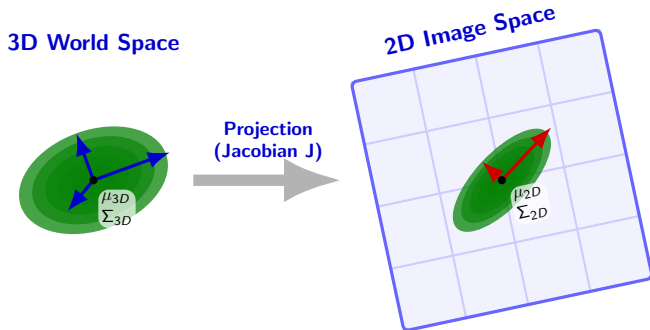
- ▶ The algorithm mathematically rebuilds the 3D geometric shape.
- ▶ It combines the scaling matrix  $S$  and rotational matrix  $R$  to form the full 3D covariance matrix:

$$\Sigma_{3D} = RSS^T R^T$$

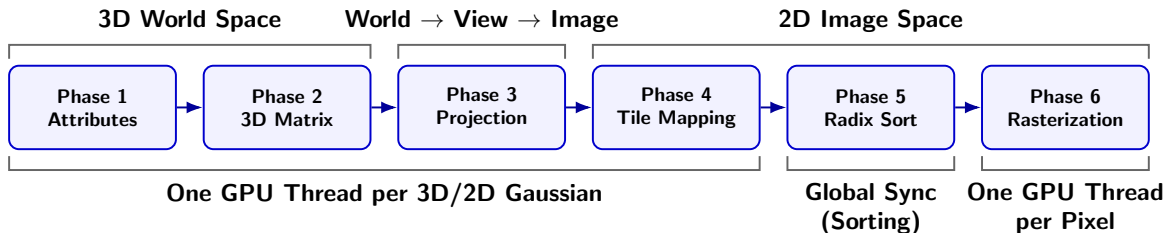
## Phase 3: Projection from 3D to 2D

- ▶ Converts 3D Gaussians (ellipsoids) in world space into flat 2D Gaussians (ellipses) in image space.
- ▶ Uses a linear affine approximation to keep the 2D Gaussian elliptic (Jacobian  $J$ ).
- ▶ The 3D matrix is rotated into view space via camera matrix  $W_{rot}$  and then projected:

$$\Sigma_{2D} = JW_{rot}\Sigma_{3D}W_{rot}^TJ^T$$



# Forward Pass: Pipeline Architecture



### Phase 4: Tile Mapping (Preparing Radix Sort)

- ▶ The target output image is divided into a grid of  $16 \times 16$  pixel tiles.
- ▶ The algorithm calculates a 2D bounding box to map exactly which tiles a given 2D Gaussian covers.
- ▶ Assigns each 2D Gaussian to a tile. If overlapping, assigns to all affected tiles.

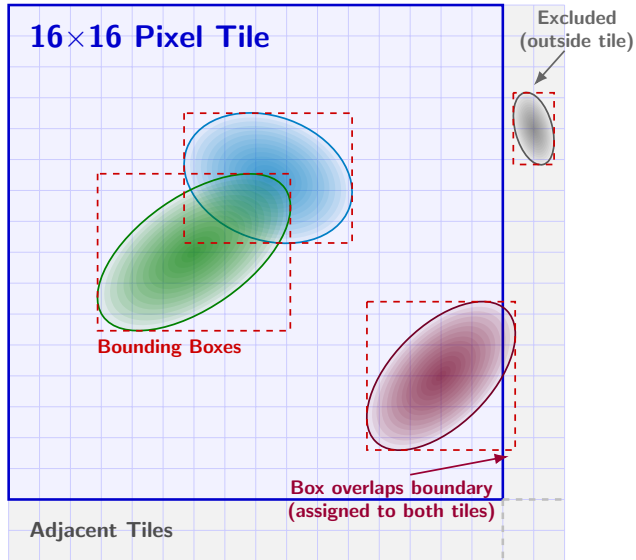
### Phase 4: Tile Mapping (Preparing Radix Sort)

- ▶ The target output image is divided into a grid of  $16 \times 16$  pixel tiles.
- ▶ The algorithm calculates a 2D bounding box to map exactly which tiles a given 2D Gaussian covers.
- ▶ Assigns each 2D Gaussian to a tile. If overlapping, assigns to all affected tiles.

### Phase 5: Global Radix Sort

- ▶ The GPU waits until all per-Gaussian computations are finished.
- ▶ A highly optimized Radix Sort is executed, organizing the 2D Gaussians for each tile strictly by depth (closest to farthest).

## Forward Pass: Phases 4 & 5



### Phase 6: Tile-Based Rasterization (Splatting)

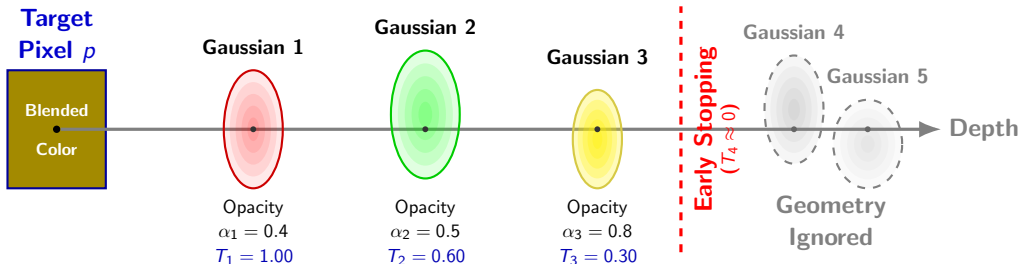
- ▶ Operating in 2D image space, the GPU assigns one processing thread per pixel.
- ▶ **Alpha Compositing:** Pixels accumulate color by blending the sorted list of overlapping 2D Gaussians front-to-back:

$$C_p = \sum_{i=1}^N c_i \alpha_i T_i, \quad \text{with} \quad T_i = \prod_{j=1}^{i-1} (1 - \alpha_j) \quad \text{the transmittance}$$

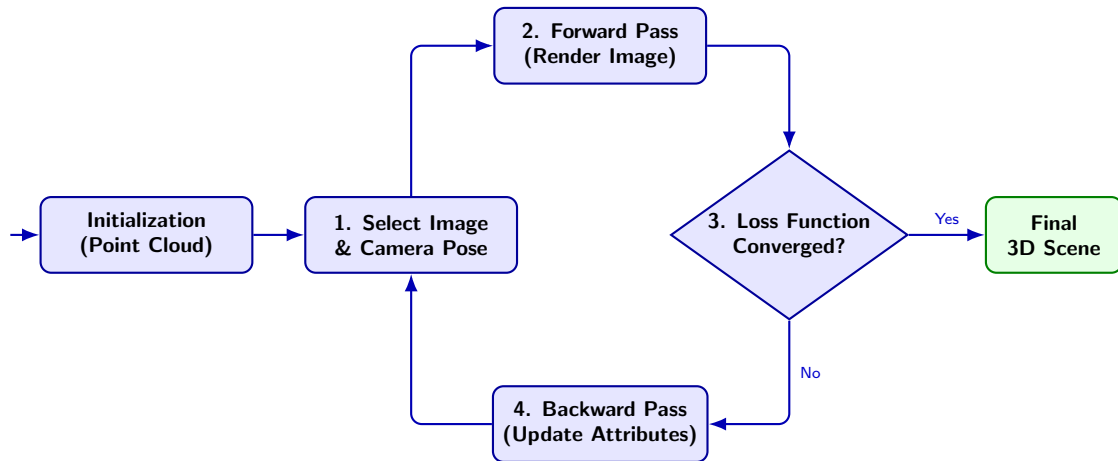
## Phase 6: Tile-Based Rasterization (Splatting)

- ▶ Operating in 2D image space, the GPU assigns one processing thread per pixel.
- ▶ **Alpha Compositing:** Pixels accumulate color by blending the sorted list of overlapping 2D Gaussians front-to-back:

$$C_p = \sum_{i=1}^N c_i \alpha_i T_i, \quad \text{with} \quad T_i = \prod_{j=1}^{i-1} (1 - \alpha_j) \quad \text{the transmittance}$$

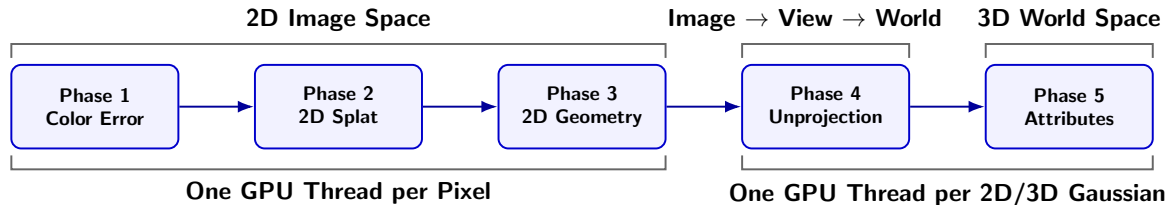


# The Training Cycle



- 1 Background
- 2 3D Gaussian Data Structure
- 3 Training Procedure**
  - Point Cloud Initialization
  - Forward Pass
  - **Backward Pass**
- 4 Novel-View Synthesis
- 5 Summary and Discussion

# Backward Pass: Pipeline Architecture



## The Loss Function

- ▶ Training compares the rendered image with the original input image using a weighted combination of  $\mathcal{L}_1$  loss and D-SSIM.
- ▶  $\mathcal{L}_1$  calculates the absolute pixel-wise error, while D-SSIM acts as a structural regularizer:

$$L = (1 - \lambda) \cdot \mathcal{L}_1 + \lambda \cdot \mathcal{L}_{D-SSIM}, \quad \text{with } \lambda = 0.2$$

## The Loss Function

- ▶ Training compares the rendered image with the original input image using a weighted combination of  $\mathcal{L}_1$  loss and D-SSIM.
- ▶  $\mathcal{L}_1$  calculates the absolute pixel-wise error, while D-SSIM acts as a structural regularizer:

$$L = (1 - \lambda) \cdot \mathcal{L}_1 + \lambda \cdot \mathcal{L}_{D-SSIM}, \quad \text{with } \lambda = 0.2$$

## Phase 1: Loss to Pixel Color Correction

- ▶ The gradient calculates the required color correction per pixel ( $C_p$ ).
- ▶ A positive gradient indicates an overestimation of color, negative indicates underestimation, and the magnitude dictates the degree of adjustment required:

$$\frac{\partial L}{\partial C_p} = \frac{1 - \lambda}{N} \cdot \text{sign}(C_p - O_p) + \lambda \frac{\partial \mathcal{L}_{D-SSIM}}{\partial C_p}$$

### Phase 2: Pixel Color Correction to 2D Splat

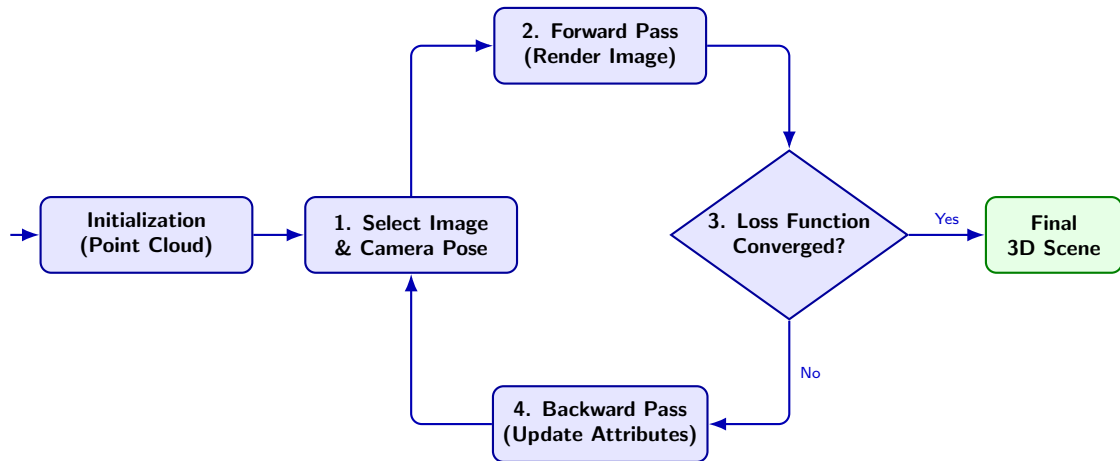
- ▶ The algorithm works alpha compositing backwards, flowing the pixel gradient into the evaluated opacity ( $\alpha_i$ ) and color ( $c_i$ ) of each 2D Gaussian in the tile.
- ▶ **Color Gradient:** Uses the chain rule to update the Gaussian's color based on its contribution (transmittance  $T_i$  and opacity  $\alpha_i$ ):

$$\text{with } C_p = \sum_{i=1}^N c_i \alpha_i T_i : \quad \frac{\partial L}{\partial c_i} = \frac{\partial L}{\partial C_p} \cdot \frac{\partial C_p}{\partial c_i} = \frac{\partial L}{\partial C_p} \cdot (\alpha_i T_i)$$

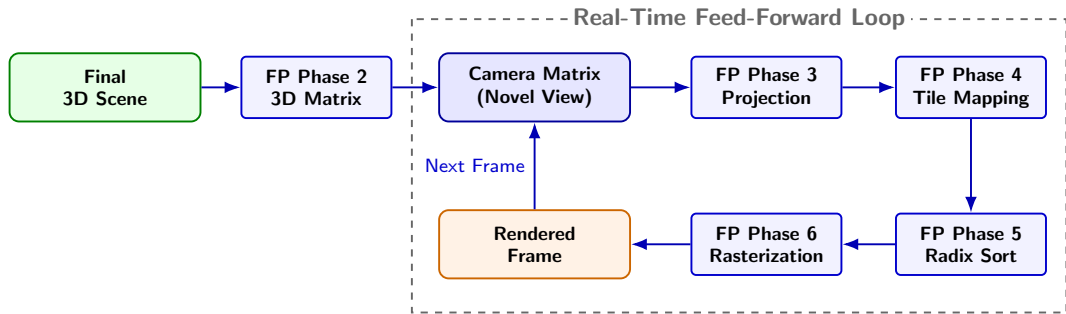
## Adaptive Density Control

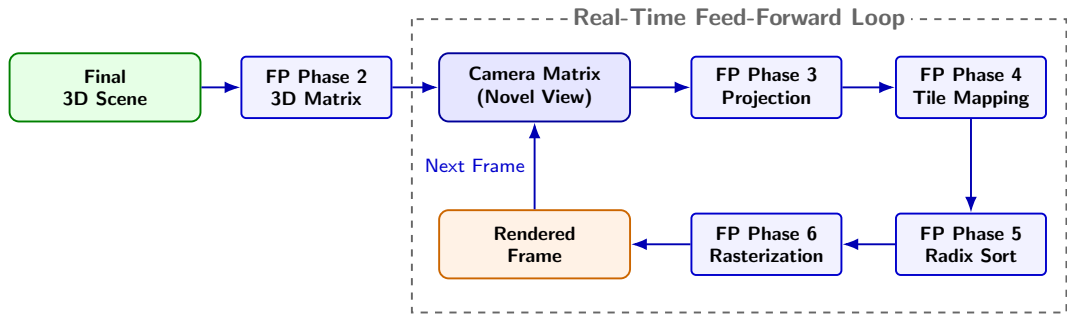
- ▶ Steps in every 100 iterations to counter over-reconstruction (covering too much geometry) and under-reconstruction (too few Gaussians).
- ▶ **Splitting:** Over-reconstructing Gaussians are split into two smaller ones.
- ▶ **Cloning:** Under-reconstructing Gaussians are cloned.
- ▶ **Pruning & Opacity Reset:** 3D Gaussians with extremely low opacity or unbound scale are routinely deleted.

# The Training Cycle



- 1 Background
- 2 3D Gaussian Data Structure
- 3 Training Procedure
  - Point Cloud Initialization
  - Forward Pass
  - Backward Pass
- 4 Novel-View Synthesis**
- 5 Summary and Discussion





- ▶ Because the scene geometry is finalized, the backward pass, Adaptive Density Control, and other components are completely disabled.
- ▶ Freed from this overhead and running at the raw speed of the tile-based rasterizer, 3DGS quickly reaches real-time rendering levels.

- 1 Background
- 2 3D Gaussian Data Structure
- 3 Training Procedure
  - Point Cloud Initialization
  - Forward Pass
  - Backward Pass
- 4 Novel-View Synthesis
- 5 Summary and Discussion**

- ▶ 3D Gaussian Splatting is a learnable and explicit radiance field representation for 3D Reconstruction, abandoning neural networks.
- ▶ Kerbl et al. combined multiple existing approaches and technologies, added new ideas, and mapped them onto modern GPU hardware.
- ▶ We analyzed the training procedure showing in detail how Kerbl. et al made it possible to have real-time rendering of reconstructed 3D scenes for the first time.

Thank you for your attention. I am looking forward to taking your questions.

**6** Videos

7 Backpropagation Gradient Flow

8 Backpropagation Calculations

9 Selected References

## Training in Progress

<https://youtu.be/F9rgbvhSad8?t=550>



<https://youtu.be/yd40zUnr7eQ>



## Real-Time Rendering

Example from the original paper's project website

<https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/content/videos/bicycle.mp4>



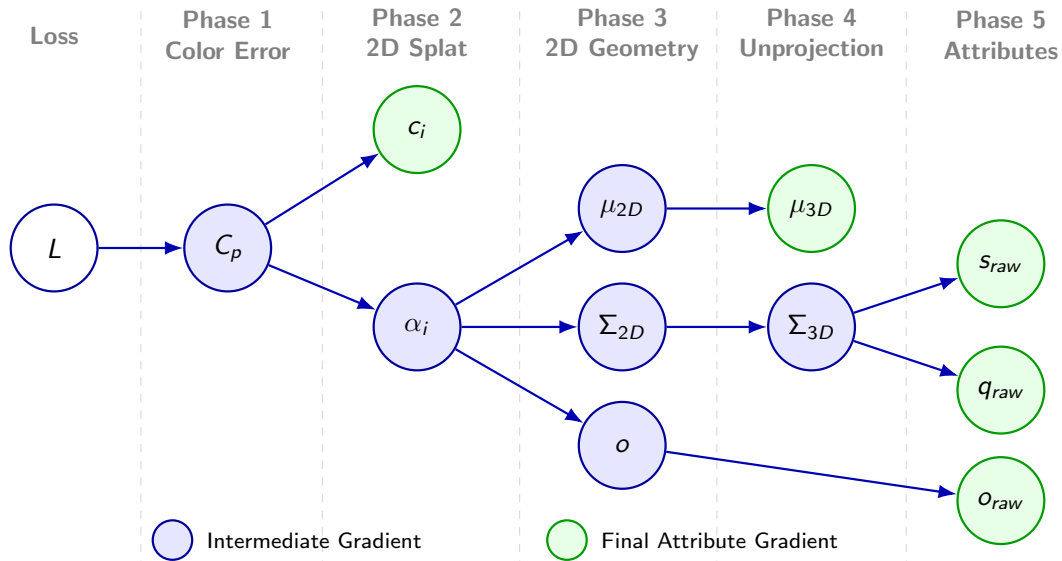
6 Videos

**7** Backpropagation Gradient Flow

8 Backpropagation Calculations

9 Selected References

# Backward Pass: Gradient Flow Overview



6 Videos

7 Backpropagation Gradient Flow

**8 Backpropagation Calculations**

9 Selected References

### Phase 3: 2D Splat to 2D Geometry (Opacity & Position)

- ▶ We recall the simplified opacity distribution:  $\alpha_i(p) = o \cdot e^E$ , where  $E$  is the exponent derived from the pixel distance  $\Delta$  and the inverse 2D covariance matrix.
- ▶ **Base Opacity ( $o$ ):** The derivative scales directly with the distance from the center of the 2D Gaussian ( $e^E$ ):

$$\frac{\partial L}{\partial o} = \frac{\partial L}{\partial \alpha_i} \cdot e^E$$

- ▶ **Position ( $\mu_{2D}$ ):** Let vector  $v = \Sigma_{2D}^{-1} \Delta p$ . This vector points directly from the 2D Gaussian's center to the pixel:

$$\frac{\partial L}{\partial \mu_{2D}} = \frac{\partial L}{\partial \alpha_i} \cdot \alpha_i(p) \cdot v$$

- ▶ If the evaluated opacity gradient is positive, the center is effectively pulled towards the pixel.

### Phase 3: 2D Splat to 2D Geometry (Shape)

- ▶ **2D Covariance Matrix ( $\Sigma_{2D}$ ):** We must calculate how the opacity at pixel  $p$  changes when the 2D shape of the Gaussian changes.
- ▶ This uses the same vector  $v$  from the position derivative, multiplied by its own transpose:

$$\frac{\partial L}{\partial \Sigma_{2D}} = \frac{\partial L}{\partial \alpha_i} \cdot \frac{1}{2} \alpha_i(p) \cdot (v \cdot v^T)$$

- ▶ Multiplying the vector by its transpose results in a matrix gradient.
- ▶ If the evaluated opacity gradient is positive, this matrix gradient causes the 2D Gaussian to stretch itself towards the pixel; otherwise, it causes it to contract.

### Phase 4: 2D Geometry to 3D Geometry

- ▶ The GPU thread model changes to running per-Gaussian to unproject the 2D data back into 3D.
- ▶ This utilizes the multi-variable chain rule and the transposes of the forward projection matrices ( $U = JW_{rot}$ ).
- ▶ **3D Position ( $\mu_{3D}$ ):** The gradient passes through view space ( $t$ ) back to world space:

$$\frac{\partial L}{\partial \mu_{3D}} = W_{rot}^T J^T \frac{\partial L}{\partial \mu_{2D}} = U^T \left( \frac{\partial L}{\partial \mu_{2D}} \right)$$

- ▶ **3D Covariance ( $\Sigma_{3D}$ ):** The final gradient unprojects cleanly using matrix  $U$  on both sides:

$$\frac{\partial L}{\partial \Sigma_{3D}} = U^T \left( \frac{\partial L}{\partial \Sigma_{2D}} \right) U$$

### Phase 5: 3D Geometry to Matrix Components

- ▶ Gradients flow through the covariance matrix assembly ( $\Sigma_{3D} = MM^T$  with  $M = RS$ ) into scale ( $S$ ) and rotation ( $R$ ).
- ▶ First, the algorithm derives the intermediate matrix  $M$ :

$$\frac{\partial L}{\partial M} = 2 \left( \frac{\partial L}{\partial \Sigma_{3D}} \right) M$$

- ▶ Then, it splits this gradient into the separate components:

$$\frac{\partial L}{\partial S} = R^T \left( \frac{\partial L}{\partial M} \right) \quad \text{and} \quad \frac{\partial L}{\partial R} = \left( \frac{\partial L}{\partial M} \right) S$$

- ▶ The scale gradient ( $\frac{\partial L}{\partial s}$ ) is extracted from the diagonal of  $\frac{\partial L}{\partial S}$ , while the quaternion gradient ( $\frac{\partial L}{\partial q}$ ) aggregates the partial derivatives of  $R$ .

### Phase 5: Backpropagating to Latent Attributes

- ▶ To close the loop, the gradients pass backwards through their respective activation functions to reach the raw, latent space variables.
- ▶ **Scale (Exponential):** Backpropagating through exp requires multiplying by  $s$ :

$$\frac{\partial L}{\partial s_{raw}} = \frac{\partial L}{\partial s} \cdot s$$

- ▶ **Rotation (Normalization):** Uses the Jacobian matrix ( $J_{||}$ ) of the normalization function, with  $J_{||} = \frac{I - qq^T}{||q_{raw}||}$ :

$$\frac{\partial L}{\partial q_{raw}} = J_{||} \cdot \frac{\partial L}{\partial q}$$

- ▶ **Base Opacity (Sigmoid):** Backpropagating through the sigmoid function:

$$\frac{\partial L}{\partial o_{raw}} = \frac{\partial L}{\partial o} \cdot o(1 - o)$$

6 Videos

7 Backpropagation Gradient Flow

8 Backpropagation Calculations

9 Selected References

- [1] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkuehler, and George Drettakis.  
3D Gaussian Splatting for Real-Time Radiance Field Rendering.  
*ACM Transactions on Graphics*, 42(4):1–14, August 2023.
- [2] Christoph Lassner and Michael Zollhöfer.  
Pulsar: Efficient Sphere-based Neural Rendering.  
In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1440–1449, June 2021.
- [3] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud.  
DUST3R: Geometric 3D Vision Made Easy.  
In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20697–20709, June 2024.
- [4] Lee Alan Westover.  
*Splatting: A Parallel, Feed-Forward Volume Rendering Algorithm*.  
PhD thesis, The University of North Carolina at Chapel Hill, 1991.

- [5] Matthias Zwicker, Hanspeter Pfister, Jeroen van Baar, and Markus Gross.  
EWA Volume Splatting.  
In *Proceedings Visualization, 2001, VIS '01*, pages 29–36, San Diego, CA, USA, 2001. IEEE.
- [6] Matthias Zwicker, Hanspeter Pfister, Jeroen van Baar, and Markus Gross.  
Surface Splatting.  
In *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH '01*, pages 371–378, New York, NY, USA, 2001. Association for Computing Machinery.